

IconPackHub

Документация кастомизации пака

Полное описание всех компонентов, настроек и API
для восстановления работы проекта

Версия проекта: актуальная (29.06.2026)

1. Обзор системы кастомизации

Система кастомизации пака в IconPackHub позволяет пользователям изменять внешний вид иконок в паке: цвета, толщину линий, размер, фон, тени, анимации и многое другое. Кастомизация работает через конфигурационный объект **CustomConfig**, который применяется к SVG-иконкам через функцию **renderSvg()**. Результат можно скачать как отдельные SVG-файлы или как ZIP-архив со всем паком. Кастомизированные паки можно сохранить в личный кабинет и скачать позже.

Архитектура системы состоит из трёх основных слоёв: (1) фронтенд-компонент **Customize** (src/views/customize.tsx) — интерфейс с панелями настроек и превью иконок в реальном времени; (2) SVG-рендерер (src/lib/svg.ts) — серверный и клиентский движок, который принимает сырой SVG из базы данных и конфиг CustomConfig, а возвращает готовый SVG с применёнными стилями, фонами, градиентами, тенями и SMIL-анимациями; (3) API-слой — набор эндпоинтов для рендеринга, сохранения, скачивания и управления кастомизированными паками и палитрами.

Ключевой принцип: все иконки в базе используют **currentColor** вместо хардкода цветов. Это позволяет кастомизатору подменять currentColor на любой выбранный пользователем цвет, включая градиенты. Иконки на основе заливки (fill-based) и обводки (stroke-based) обрабатываются разными алгоритмами для корректного применения цветов.

2. CustomConfig — полная спецификация

CustomConfig — центральный тип, описывающий все параметры кастомизации одной иконки. Он определён в src/lib/svg.ts и используется во всём проекте: от фронтенд-компонента Customize до API-эндпоинтов скачивания и сохранения. Ниже приведена полная таблица всех полей с типами, значениями по умолчанию и описанием.

Поле	Тип	По умолчанию	Описание
color	string	#0F172A	Основной цвет иконки (заменяет currentColor)
color2	string	#38BDF8	Вторичный цвет для режима duotone
strokeWidth	number	1.75	Толщина обводки (px)
size	number	24	Размер итогового SVG (width/height в px)
padding	number	0	Отступ вокруг иконки (0..8)
background	BgShape	'none'	Форма фона: none circle square hexagon diamond
bgColor	string	#F1F5F9	Цвет фона (если background не none)
rotation	number	0	Угол поворота иконки (градусы)
mode	'mono' 'duotone'	'mono'	Режим раскраски: моно или двухцветный
colorGradient	boolean	false	Включить градиент для основного цвета
colorGradientStops	{offset, color}[]	[[{0, '#0F172A'}, {100, '#38BDF8'}]]	Стопы градиента цвета
gradientAngle	number	135	Угол градиента цвета (градусы)
bgGradient	boolean	false	Включить градиент фона

Поле	Тип	По умолчанию	Описание
bgGradientStops	{offset, color}[]	[[{0,'#F1F5F9'},{100,'#CBD5E0'}]]	Стопы градиента фона
bgGradientAngle	number	135	Угол градиента фона (градусы)
strokeStyle	StrokeStyle	'solid'	Стиль обводки: solid dashed dotted
lineCap	LineCap	'round'	Окончание линий: round square butt
lineJoin	LineJoin	'round'	Соединение линий: round miter bevel
dashArray	string	"	Вычисленный dash-array (из strokeStyle + strokeWidth)
opacity	number	1	Прозрачность (0..1)
shadowType	ShadowType	'none'	Тень: none shadow glow
shadowColor	string	#000000	Цвет тени/свечения
shadowBlur	number	3	Размытие тени (stdDeviation)
shadowOffsetX	number	1	Смещение тени по X
shadowOffsetY	number	1	Смещение тени по Y
animation	AnimType	'none'	Тип анимации (см. раздел 5)
animDuration	number	2	Длительность анимации (секунды)
animEasing	EasingType	'ease'	Плавность: ease linear ease-in ease-out ease-in-out
animDelay	number	0	Задержка старта анимации (секунды)
animIterations	number	0	Число повторов (0 = бесконечно)
exportFormat	ExportFormat	'svg'	Формат экспорта: svg png react vue

3. Строковые типы (перечисления)

В проекте определено 8 перечисляемых типов, которые используются в CustomConfig. Каждый тип ограничивает допустимые значения соответствующего поля конфигурации. Эти типы импортируются из src/lib/svg.ts и используются в компоненте Customize для валидации пользовательского ввода и построения UI-элементов (выпадающие списки, кнопки).

Тип	Значения	Описание
StrokeStyle	solid dashed dotted	Стиль штриха линии
LineCap	round square butt	Окончание линий (stroke-linecap)
LineJoin	round miter bevel	Соединение линий (stroke-linejoin)
BgShape	none circle square hexagon diamond	Форма подложки иконки
ShadowType	none shadow glow	Тип тени или свечения
AnimType	none spin pulse bounce shake wobble swing float slide none	Тип анимации иконки

Тип	Значения	Описание
EasingType	ease linear ease-in ease-out ease-in-out	Функция плавности анимации
ExportFormat	svg png react vue	Целевой формат экспорта

4. SVG-рендерер renderSvg()

Функция `renderSvg(innerSvg, viewBox, cfg)` — ядро системы кастомизации. Она принимает сырое SVG-тело иконки из базы данных (без обёртки `<svg>`), `viewBox` (обычно `'0 0 24 24'`) и конфиг `CustomConfig`, а возвращает полностью сформированный SVG с применёнными стилями. Функция реализована в `src/lib/svg.ts` и работает как на клиенте (превью), так и на сервере (API-эндпоинты скачивания и сохранения).

4.1 Алгоритм рендеринга

Рендеринг выполняется в 8 этапов, каждый из которых последовательно модифицирует SVG-вывод. Порядок важен: например, цвет применяется до `stroke-width`, а фон добавляется перед тенями.

Этап	Описание
1. Применение цвета	Замена <code>currentColor</code> на <code>cfg.color</code> или <code>url(#gradient)</code> . В <code>duotone</code> -режиме элементы чередуются между <code>color</code> и <code>stroke-color</code> .
2. Stroke-width	Установка толщины обводки. Если <code>stroke-width</code> отсутствует — добавляется.
3. Dash-array	Вычисление штрих-паттерна из <code>strokeStyle</code> и <code>strokeWidth</code> : <code>dashed = strokeWidth*4 + strokeWidth</code> .
4. Line-cap / Line-join	Установка <code>stroke-linecap</code> и <code>stroke-linejoin</code> на все SVG-элементы.
5. Фон (background)	Добавление фигуры фона (<code>circle</code> , <code>square</code> , <code>hexagon</code> , <code>diamond</code>) с заливкой <code>bgColor</code> или градиента.
6. Тень / Свечение	Добавление SVG-фильтра <code>feDropShadow</code> (для <code>shadow</code>) или <code>feGaussianBlur</code> + <code>feMerge</code> (для <code>glow</code>).
7. Прозрачность	Обёртка в <code><g opacity='...'></code> ; если <code>opacity</code> меньше 1.
8. Анимация (SMIL)	Генерация <code>animateTransform</code> или <code>animate</code> элементов. Для <code>pulse</code> — двойной <code>translate(12,12)/translate(0,0)</code> .

4.2 Структура сгенерированного SVG

Итоговый SVG имеет вложенную структуру обёрток `<g>`, где каждый слой отвечает за свою настройку. Порядок вложения (снаружи внутрь): `<svg>` переходит в `defs` (градиенты, фильтры), затем `bgShape` (фон), `opacityWrapper`, `filterWrapper` (тень/свечение), `paddingWrapper` (отступ), `centerWrapper` (для `pulse`), SMIL `target <g>`, `rotationWrapper`, и наконец `body (paths)`. Такая архитектура гарантирует, что анимации, тени и повороты взаимодействуют корректно и не конфликтуют друг с другом.

5. Система анимаций (SMIL)

Анимации реализованы через SMIL (Synchronized Multimedia Integration Language) — встроенный в SVG стандарт, который работает нативно в браузере без CSS. Это критически важно, поскольку иконки рендерятся через `dangerouslySetInnerHTML` в React, где CSS-анимации не работают.

применяются. SMIL-элементы (animateTransform, animate) встраиваются прямо в SVG-разметку и работают автономно.

Анимация	SMIL-элемент	Описание
spin	animateTransform type='rotate'	Плавное вращение на 360 градусов вокруг центра (12,12). Всегда linear
pulse	animateTransform type='scale'	Пульсация масштаба 1 - 1.15 - 1 с центром в (12,12). Требуется translate
bounce	animateTransform type='translate'	Подпрыгивание: смещение по Y 0 - -2 - 0.
shake	animateTransform type='translate'	Тряска: горизонтальное смещение 0 - -1.5 - 0 - 1.5 - 0, 5 keyTimes, 4 и
wobble	animateTransform type='rotate'	Покачивание: вращение 0 - -4 - 4 - 0 вокруг (12,12), 4 keyTimes.
swing	animateTransform type='rotate'	Маятник: вращение 0 - 8 - 0 вокруг (12,12).
fade	animate attributeName='opacity'	Пульсация прозрачности 1 - 0.2 - 1.
float	animateTransform type='translate'	Парение: мягкое смещение 0 - -1.5 - 0 по Y.
blink	animate (calcMode='discrete')	Моргание: дискретное переключение opacity 1 - 1 - 0 - 0 - 1.
slide	animateTransform type='translate'	Скольжение: горизонтальное 0 - 1.5 - 0 - -1.5 - 0.

5.1 Критичные правила SMIL

keySplines и количество интервалов. Атрибут keySplines должен содержать ровно (values.length - 1) записей, разделённых точкой с запятой. Несовпадение количества приводит к молчаливому игнорированию всей анимации браузером. Например, shake с 5 values требует 4 keySplines; pulse с 3 values — 2 keySplines.

Easing-маппинг: ease переходит в '0.42 0 0.58 1'; ease-in переходит в '0.42 0 1 1'; ease-out переходит в '0 0 0.58 1'; ease-in-out переходит в '0.42 0 0.58 1'. Для linear используется calcMode='linear' без keySplines.

Pulse и центр масштабирования. SMIL type='scale' масштабирует от начала координат (0,0), а не от центра. Для корректной работы используется обёртка: translate(12,12) переходит в scale-анимацию, затем translate(-12,-12). Это делает масштабирование от центра иконки.

6. Фронтенд: компонент Customize

Компонент Customize (src/views/customize.tsx) — главный UI кастомизации. Он отображает превью всех иконок пака с применением конфига в реальном времени, а также три панели настроек: цвет, стиль и анимация. Компонент поддерживает три режима области применения настроек: all (ко всем иконкам), single (к одной), multi (к выбранным).

6.1 Режимы области применения (ScopeMode)

Режим	Описание
all	Настройки применяются ко всем иконкам пака одновременно. Используется по умолчанию.
single	Настройки применяются только к одной выбранной иконке. Иконка выбирается кликом.

Режим	Описание
multi	Настройки применяются к нескольким выбранным иконкам. Выбор через Ctrl+клик или чекбоксы.

6.2 Панели настроек (ControlTab)

Интерфейс разделён на три вкладки: **color** (цвет, градиент, режим mono/duotone, палитры), **style** (толщина обводки, размер, отступ, фон, поворот, прозрачность, стиль линий, тень/свечение) и **animation** (тип анимации, длительность, плавность, задержка, повтор). Каждая вкладка управляет соответствующей группой полей CustomConfig.

6.3 Встроенные палитры (BUILTIN_PALETTES)

Компонент содержит 6 предустановленных палитр, которые пользователь может выбрать одним кликом. Каждая палитра задаёт color1, color2, bgColor1, bgColor2, isGradient, isBgGradient, gradientAngle и mode. Палитры: Dark (моно, тёмный на светлом), Neon (дуотон, градиент фиолетовый на тёмном фоне), Sunset (дуотон, тёплый градиент), Ocean (моно, синий), Forest (моно, зелёный), Candy (дуотон, розово-голубой).

6.4 Пользовательские палитры (UserPalette)

Пользователи могут создавать собственные палитры через API /api/palettes. Палитра сохраняется в базу данных (модель UserPalette) и привязывается к userId. Структура UserPalette совпадает с BUILTIN_PALETTES: nameRu, nameEn, color1, color2, bgColor1, bgColor2, isGradient, isBgGradient, gradientAngle, mode. При применении палитры её значения мапятся на поля CustomConfig.

6.5 Стоимость кастомизации

Сохранение кастомизированного пака стоит **5 кредитов** за операцию (константа COST = 5 в customize.tsx). Скачивание кастомизированного пака бесплатно для подписчиков, но ограничено 3 скачиваниями в месяц для бесплатных пользователей. Рендеринг превью на клиенте бесплатен — кредиты списываются только при сохранении.

7. База данных: модели кастомизации

Система кастомизации использует 5 моделей Prisma (SQLite), которые хранят конфиги, пользовательские иконки, паки, палитры и связи между ними. Схема определена в prisma/schema.prisma.

7.1 CustomIcon

Хранит отдельную кастомизированную иконку. Поля: id, userId, baseIconId (ссылка на оригинальную Icon), basePackId, name, config (JSON-строка с полным CustomConfig), svgSnapshot (готовый отрендеренный SVG). svgSnapshot кэшируется при сохранении, чтобы не рендерить повторно при просмотре/скачивании.

7.2 CustomPack

Пользовательский пак — набор кастомизированных иконок. Поля: id, userId, name, basePackId (опционально — если пак создан из одного стандартного пака), createdAt. Связь с иконками через промежуточную таблицу CustomPackIcon.

7.3 CustomPackIcon

Связующая таблица между CustomPack и CustomIcon. Поля: id, customPackId, customIconId (опционально), baseIconId (опционально), sortOrder (порядок иконки в паке). Позволяет одному CustomIcon входить в несколько CustomPack.

7.4 UserPalette

Пользовательская цветовая палитра. Поля: id, userId, nameRu, nameEn, color1, color2, bgColor1, bgColor2, isGradient, isBgGradient, gradientAngle, mode ('mono' или 'duotone'), createdAt. Палитра применяется как пресет — её значения подставляются в CustomConfig.

8. API-эндпоинты кастомизации

Все API-эндпоинты реализованы как Next.js Route Handlers (App Router). Авторизация — через заголовок x-user-email (идентификация по email).

Эндпоинт	Параметры	Описание
POST /api/customize	svg, viewBox, cfg	Рендер SVG на сервере. Принимает сырое SVG-тело и
GET /api/palettes	—	Список пользовательских палитр (требуется авторизац
POST /api/palettes	nameRu, nameEn, color1, color2, bgColor1, bgColor2, isGradient, isBgGradient, gradientAngle	Создание палитры
DELETE /api/palettes/[id]	—	Удаление палитры по ID (с проверкой владельца).
POST /api/packs/save	name, items: [{iconId, cfg?}]	Сохранение кастомизированного пака в ЛК. Бесплатно
GET /api/packs/saved	—	Список сохранённых паков пользователя с иконками и
DELETE /api/packs/saved/[id]	—	Удаление сохранённого пака (с проверкой владельца,
GET /api/download/pack?slug=xxx	slug, cfg (опц.), cfgMap (опц.)	Скачать пак как ZIP. cfg — общий конфиг для всех икон
POST /api/download/pack	slug, cfg?, cfgMap?	POST-версия скачивания пака (для больших конфигов,
GET /api/download/icon?id=xxx	id, cfg (опц.)	Скачать одну иконку как SVG. cfg применяется через re
POST /api/download/build	name, items: [{iconId, cfg?}]	Скачать сборку иконок как ZIP. Проверяет лимит: 3 бес
GET /api/download/saved/[id]	—	Скачать сохранённый пак как ZIP из кэша svgSnapshot.

8.1 Параметр cfgMap (per-icon конфиг)

При скачивании пака (GET/POST /api/download/pack) можно передать параметр cfgMap — JSON-объект вида {iconId: CustomConfig}. Это позволяет применить разные настройки к разным иконкам в одном паке. Если иконка есть в cfgMap — используется её конфиг; если нет — используется глобальный cfg (если задан) или дефолтный рендеринг. Это соответствует режимам single/multi на фронтенде.

8.2 Лимит скачиваний

Система лимитирует бесплатные скачивания: 3 сборки (build) в месяц для пользователей без активной подписки. Счётчик `freeBuildsUsed` сбрасывается при смене месяца (проверка через `isDifferentMonth`). Подписчики (с активной `Subscription`) скачивают без ограничений. Неавторизованные пользователи могут скачивать без лимита (для обратной совместимости).

9. Компонент Builder (сборка пака)

Builder (`src/views/builder.tsx`) — страница сборки собственного пака из отдельных иконок. Пользователь добавляет иконки из каталога через кнопку '+' на карточке иконки. Состояние сборки хранится в `BuildContext` (`src/lib/build-store.tsx`) — React Context с персистентностью в `localStorage` под ключом 'build'.

Builder предоставляет упрощённую панель кастомизации с базовыми настройками: цвет, толщина обводки, размер (16/20/24/32/48 px), фон (`none/circle/square`) и цвет фона. Для полноценной кастомизации пользователь переходит в `Customize`. Из Builder можно скачать сборку (POST `/api/download/build`) или сохранить в 'Мои паки' (POST `/api/packs/save`).

10. Навигация и маршруты

Навигация реализована через тип `View` (`src/lib/navigation.ts`), который маппит виртуальные маршруты на реальные URL. Для кастомизации используются два маршрута: `{name: 'customize', packSlug, iconId?}` переходит в `/catalog/[slug]/customize[?icon=...]` и `{name: 'pack', slug}` переходит в `/catalog/[slug]`. Страница кастомизации рендерится компонентом `CustomizePage` (`src/app/catalog/[slug]/customize/page.tsx`), который извлекает `slug` из параметров и `iconId` из `searchParams`.

View	Параметры	URL	Описание
customize	packSlug, iconId (опц.)	/catalog/[slug]/customize[?icon=...]	Страница кастомизации пака
pack	slug	/catalog/[slug]	Просмотр пака с кнопкой 'Кастомизировать'
builder	—	/builder	Сборка пака из отдельных иконок
my-packs	—	/my-packs	Сохранённые кастомизированные паки

11. ZIP-генератор makeZip()

Функция `makeZip(files)` из `src/lib/svg.ts` генерирует ZIP-архив без сжатия (store mode) напрямую в памяти, без внешних зависимостей. Реализует минимум спецификации PKZIP: Local file header + data + Central directory + End of central directory. CRC32 вычисляется вручную. Результат конвертируется в base64, затем в Buffer для отдачи как HTTP-ответ с Content-Type: application/zip.

Архив содержит: отдельные SVG-файлы (`packSlug/iconSlug.svg`), `sprite.svg` (symbol для каждого icon) и `README.md` с названием и описанием пака. При кастомизированном скачивании SVG-файлы уже содержат применённый конфиг.

12. Seed-данные паков

Исходные паки и иконки определены в `src/lib/packs-data.ts` как массив `PACKS: PackDef[]`. Каждый пак содержит `slug`, `nameRu/En`, `descRu/En`, `category`, `style`, `tags`, `isFree`, `priceCredits` и массив `icons: IconDef[]`. Каждая `IconDef` содержит `slug`, `nameRu/En`, `keywords` и `svg` (сырое SVG-тело). Все иконки используют `currentColor` для поддержки кастомизации.

Категории паков: `languages` (языки программирования), `frameworks`, `devops`, `design`, `medical`, `realestate`, `law`, `finance`, `education`, `weather`, `social`, `commerce` и др. Стили: `outline`, `filled`, `duotone`. Seed-данные загружаются через API `/api/admin/seed` или CLI-скрипт `seed-turso.ts`. Регенерация: `node scripts/build-seed.js`.

13. Ключевые файлы проекта

Ниже перечислены все файлы, относящиеся к системе кастомизации, с кратким описанием их роли. Этих файлов достаточно для полного восстановления функциональности.

Файл	Роль
<code>src/lib/svg.ts</code>	SVG-рендерер, типы <code>CustomConfig/DEFAULT_CONFIG</code> , <code>makeZip()</code> , <code>CRC32</code>
<code>src/views/customize.tsx</code>	UI кастомизации: панели настроек, превью, палитры, <code>score</code> -режимы
<code>src/views/pack-view.tsx</code>	Просмотр пака с кнопкой 'Кастомизировать'
<code>src/views/builder.tsx</code>	Сборка пака из отдельных иконок с упрощённой кастомизацией
<code>src/views/my-packs.tsx</code>	Список сохранённых кастомизированных паков
<code>src/components/icon-view.tsx</code>	Компонент рендеринга иконки (использует <code>renderSvg</code>)
<code>src/lib/build-store.tsx</code>	React Context + <code>localStorage</code> для сборки иконок
<code>src/lib/user-store.tsx</code>	React Context для авторизации и данных пользователя
<code>src/lib/navigation.ts</code>	Маршрутизация: <code>View type</code> , <code>viewToHref()</code> , <code>useNav()</code>
<code>src/lib/packs-data.ts</code>	Seed-данные: <code>PackDef[]</code> , <code>IconDef[]</code> , <code>PACKS</code>
<code>src/app/api/customize/route.ts</code>	POST <code>/api/customize</code> — серверный рендеринг SVG
<code>src/app/api/palettes/route.ts</code>	GET/POST <code>/api/palettes</code> — управление палитрами
<code>src/app/api/palettes/[id]/route.ts</code>	DELETE <code>/api/palettes/[id]</code> — удаление палитры
<code>src/app/api/packs/save/route.ts</code>	POST <code>/api/packs/save</code> — сохранение кастом-пака
<code>src/app/api/packs/saved/route.ts</code>	GET <code>/api/packs/saved</code> — список сохранённых паков
<code>src/app/api/packs/saved/[id]/route.ts</code>	DELETE <code>/api/packs/saved/[id]</code> — удаление пака
<code>src/app/api/download/pack/route.ts</code>	GET/POST — скачать пак как ZIP
<code>src/app/api/download/icon/route.ts</code>	GET — скачать одну иконку как SVG
<code>src/app/api/download/build/route.ts</code>	POST — скачать сборку иконок как ZIP
<code>src/app/api/download/saved/[id]/route.ts</code>	GET — скачать сохранённый пак как ZIP

Файл	Роль
prisma/schema.prisma	Схема БД: User, Pack, Icon, CustomIcon, CustomPack, CustomPackIcon, UserPack
src/app/catalog/[slug]/customize/page.tsx	Next.js страница кастомизации

14. Порядок восстановления работы

Для полного восстановления системы кастомизации из документации необходимо выполнить следующие шаги в указанном порядке. Каждый шаг зависит от предыдущих.

Шаг	Компонент	Действие
1	Схема БД	Восстановить prisma/schema.prisma — модели User, Pack, Icon, CustomIcon, CustomPack, CustomPackIcon, UserPack
2	Seed-данные	Восстановить src/lib/packs-data.ts с типами PackDef, IconDef и массивом PACKS. Загрузить в БД
3	SVG-рендерер	Восстановить src/lib/svg.ts: типы (StrokeStyle, LineCap, LineJoin, BgShape, ShadowType, AnimationType)
4	API-эндпоинты	Восстановить все route.ts файлы из таблицы в разделе 13. Каждый эндпоинт использует db, svg, packs-data
5	Stores	Восстановить build-store.tsx (BuildContext, localStorage 'build') и user-store.tsx (UserContext, localStorage 'user')
6	UI-компоненты	Восстановить customize.tsx (Customize), pack-view.tsx (PackView), builder.tsx (Builder), my-pack-view.tsx (MyPackView)
7	Навигация	Восстановить navigation.ts (View type, viewToHref, useNav) и страницы Next.js в src/app/
8	Тестирование	Проверить: загрузка пака из каталога, кастомизация, превью, сохранение, скачивание ZIP, удаление пака